

APOSTILA COMPLETA

Math, Random, BigDecimal, java.time & Localization

Aleatoriedade, precisão decimal, datas, fusos horários e internacionalização em Java, explicados com o estudo de caso de uma plataforma de reservas internacional.

CONTEÚDO DA APOSTILA

- 01 Introdução — Panorama da Seção
- 02 java.lang.Math — Funcionalidades da Calculadora
- 03 Randomização em Java
- 04 BigDecimal — Precisão, Escala e Representação Exata
- 05 Overview: java.time
- 06 TemporalField vs. TemporalUnit
- 07 Instant, Epoch Time e Fusos Horários
- 08 ZonedDateTime, Duration, Period e ChronoUnit.between
- 09 Localization — Introdução ao Locale
- 10 Internationalization — ResourceBundle

Sumário

01 Introdução — Panorama da Seção

Visão geral dos utilitários numéricos e de data/hora que sustentam qualquer sistema real.

02 `java.lang.Math` — Funcionalidades da Calculadora

Arredondamento, potência, raiz, overflow silencioso e os métodos `*Exact`.

03 Randomização em Java

`Math.random` vs. `Random`, geração em intervalos específicos, seeds e streams aleatórios.

04 `BigDecimal` — Precisão, Escala e Representação Exata

Como o `BigDecimal` armazena valores, `RoundingMode`, `MathContext` e propagação de escala.

05 Overview: `java.time`

Pacotes de `java.time`, armazenamento interno e o vocabulário comum `get/with/plus/is`.

06 `TemporalField` vs. `TemporalUnit`

Quando usar `ChronoField` e quando usar `ChronoUnit`, de acordo com a assinatura do método.

07 `Instant`, Epoch Time e Fusos Horários

A classe `Instant`, epoch seconds, identificadores de fuso e `ZonedDateTime` na prática.

08 `ZonedDateTime`, `Duration`, `Period` e `ChronoUnit.between`

Convertendo horários entre fusos, `TemporalAdjusters` e a diferença entre `Duration` e `Period`.

09 Localization — Introdução ao Locale

`Locale`, `NumberFormat`, `Currency` e como o Java formata datas e valores por região.

10 Internationalization — `ResourceBundle`

Como externalizar mensagens de interface para múltiplos idiomas com `ResourceBundle`.

01

Introdução — Panorama da Seção

O que será coberto: `Math`, aleatoriedade, `BigDecimal`, `java.time` e localização

Esta apostila reúne um conjunto de ferramentas da biblioteca padrão do Java que, apesar de menos comentadas do que `streams` ou `generics`, aparecem o tempo todo em sistemas de produção. Para tornar cada conceito mais concreto, todos os exemplos giram em torno de um mesmo cenário: o **Global Stay**, uma plataforma fictícia de reservas de hotel que atende hóspedes de diferentes países, em moedas e fusos horários distintos.

O conteúdo se organiza em dois grandes blocos. O primeiro revisita a classe `Math`, as opções de geração de números aleatórios e a classe `BigDecimal`, usada sempre que um cálculo precisa de exatidão decimal — como o valor cobrado por uma diária. O segundo bloco cobre o pacote `java.time` — datas, horas, instantes, durações, períodos e fusos horários — e fecha com o suporte do Java para localização e internacionalização, essencial para uma plataforma que precisa exibir preços, datas e mensagens de forma correta para hóspedes de qualquer lugar do mundo.

Estrutura da Apostila

- Capítulos 02–04: `Math`, `Random` e `BigDecimal` — utilitários numéricos de uso diário.
- Capítulos 05–08: `java.time` — datas e horas locais, campos e unidades temporais, instantes, fusos horários, durações e períodos.
- Capítulos 09–10: `Locale` e `ResourceBundle` — como o Java adapta números, datas e texto a diferentes culturas e idiomas.

REGRA PRÁTICA

Todas as classes do pacote `java.time` — `LocalDate`, `LocalTime`, `LocalDateTime`, `Instant`, `Duration`, `Period`, `ZonedDateTime` — são imutáveis e `thread-safe`. Métodos como `plus`, `minus` e `with` sempre devolvem uma nova instância; o objeto original nunca é alterado. É o mesmo padrão de imutabilidade já visto em `String`, aplicado de forma sistemática a todo o pacote.

02

java.lang.Math — Funcionalidades da Calculadora

Métodos aritméticos, científicos e constantes da classe Math

A classe Math, no pacote java.lang, funciona como uma calculadora embutida na linguagem: reúne operações que se esperaria encontrar em qualquer calculadora — soma de potências, raízes, arredondamento — além de funções científicas mais avançadas. Todo método de Math é static, e a classe não pode ser instanciada: seu único construtor é private.

Categorias de Métodos

- Aritméticos básicos: abs, max, min, pow, sqrt, cbrt.
- Arredondamento: round, ceil, floor — cada um com uma regra distinta de aproximação.
- Exponenciais e logarítmicos: exp, log, log10.
- Trigonométricos: sin, cos, tan e suas inversas (asin, acos, atan).
- Constantes: Math.PI e Math.E, ambas double e final.

No Global Stay, esses métodos aparecem o tempo todo em cálculos de ocupação e distância. Por exemplo, ao estimar a área de uma piscina circular do hotel ou ao decidir se uma reserva deve ser arredondada para cima na cobrança de uma diária parcial:

```
double raioPiscina = 6.5;
double areaPiscina = Math.PI * Math.pow(raioPiscina, 2);

int hospedesNoTurno = Math.max(quartosOcupados, reservasPendentes);
double distanciaAeroportoKm = Math.sqrt(deltaX * deltaX + deltaY * deltaY);

// diária cobrada sempre arredondada para cima em horas extras de check-out:
double horasExtras = Math.ceil(minutosAtraso / 60.0);
// já uma média de avaliações segue o arredondamento tradicional:
long notaMediaArredondada = Math.round(4.6);
```

ATENÇÃO

round, ceil e floor têm comportamentos diferentes por design: round segue o arredondamento matemático tradicional (0,5 sobe), enquanto ceil sempre sobe e floor sempre desce, independentemente da casa decimal. Escolher o método errado para uma regra de cobrança — como decidir se uma hora extra de estadia gera cobrança de uma diária inteira — é um erro sutil e comum.

Overflow Silencioso e os Métodos *Exact*

Overflow acontece quando uma operação faz um valor ultrapassar o máximo (ou o mínimo) que seu tipo consegue representar. Em Java, isso acontece silenciosamente por padrão: o código continua rodando sem lançar erro nenhum, e o valor simplesmente "dá a volta", saltando do máximo positivo

direto para o mínimo negativo do tipo. Um contador de pontos de fidelidade armazenado como `int`, se chegar perto de `Integer.MAX_VALUE` e continuar sendo incrementado, vira negativo sem qualquer aviso — algo catastrófico se esse número representa um saldo que o hóspede pode resgatar.

```
int pontosFidelidade = Integer.MAX_VALUE - 3;
for (int i = 0; i < 6; i++, pontosFidelidade++) {
    System.out.printf("%,d%n", pontosFidelidade);
}
// a partir da 4a iteracao, o saldo "vira" negativo -- overflow silencioso
```

Para os pontos do sistema em que esse tipo de estouro representaria um problema sério, Java oferece uma família de métodos em `Math` que lançam `ArithmeticException` em vez de deixar o valor estourar em silêncio: `incrementExact`, `decrementExact`, `addExact`, `subtractExact`, `multiplyExact` e `absExact`. Cada um tem a mesma semântica da operação comum, mas verifica os limites do tipo antes de retornar o resultado.

```
pontosFidelidade = Math.incrementExact(pontosFidelidade);
// lanca ArithmeticException no overflow, em vez de virar negativo
```

Usar as versões **Exact** em todo lugar seria exagero — a verificação tem um custo, e na maioria dos casos overflow não é uma possibilidade real. Mas em pontos críticos, como o saldo de pontos de um hóspede, lançar uma exceção e tratá-la explicitamente é bem mais seguro do que deixar o número virar negativo silenciosamente.

O Caso Especial de `abs`

O método `abs` ilustra bem por que overflow silencioso é perigoso mesmo em operações aparentemente inofensivas. O intervalo de `int` não é simétrico: vai de `-2147483648` até `2147483647` — o mínimo tem uma unidade de magnitude a mais que o máximo. Isso significa que não existe um `int` positivo capaz de representar o valor absoluto de `Integer.MIN_VALUE`, e `Math.abs(Integer.MIN_VALUE)` sofre overflow, devolvendo o próprio `Integer.MIN_VALUE` — um número negativo saindo de uma função que deveria sempre devolver algo não negativo.

```
int diferencaDiarias = Integer.MIN_VALUE;
System.out.println(Math.abs(diferencaDiarias)); // -2147483648 (!) -- overflow

// duas formas seguras de lidar com esse caso extremo:
Math.absExact(diferencaDiarias); // lanca ArithmeticException
Math.abs((long) diferencaDiarias); // 2147483648 -- correto, sem overflow
```

min e max — Cuidado com a Precisão de float

`Math.max` e `Math.min` recebem dois argumentos e devolvem o maior ou o menor entre eles, com quatro versões sobrecarregadas cada — para `int`, `long`, `float` e `double`. O ponto de atenção real aparece quando os dois números comparados estão muito próximos e são declarados como `float`: esse tipo carrega apenas 6 a 7 dígitos significativos de precisão, e o dígito que diferencia duas tarifas quase idênticas pode simplesmente desaparecer antes mesmo da comparação acontecer.

```
double tarifaA = 189.000002, tarifaB = 189.000005;  
System.out.println(Math.min(tarifaA, tarifaB)); // 189.000002 -- precisao preservada  
  
float fA = 189.000002f, fB = 189.000005f;  
System.out.println(Math.min(fA, fB)); // 189.0 -- float perdeu o digito menos significativo
```

DICA

Um literal numérico decimal em Java é double por padrão. Se um dos dois argumentos passados a min ou max for double, a versão double do método é usada automaticamente — basta que um único dos dois valores seja double para recuperar a precisão perdida.

03

Randomização em Java

Math.random, a classe Random, seeds e geradores pseudoaleatórios

É comum encontrar código pela internet usando Math.random para gerar valores aleatórios. Internamente, Math.random delega para uma instância de Random e chama nextDouble nela: na primeira chamada, uma única instância de java.util.Random é criada e reutilizada em todas as chamadas seguintes.

```
// padrao indireto, via Math.random:
double d = Math.random();
int quartoSorteado = (int) (Math.random() * totalQuartos) + 1;

// uso direto e explicito da classe Random:
Random random = new Random();
int quartoSorteado2 = random.nextInt(totalQuartos) + 1;
```

Esse padrão de multiplicar Math.random() pela amplitude do intervalo e somar o valor mínimo era necessário porque, até o JDK 17, Random.nextInt só aceitava um limite superior. A partir do JDK 17, quando Random passou a implementar RandomGenerator, um overload de dois argumentos resolve isso de forma direta: um limite inferior inclusivo e um superior exclusivo.

```
// antes do JDK 17 -- so existia o upper bound:
random.nextInt(totalQuartos); // 0 (inclusive) ate totalQuartos (exclusivo)

// a partir do JDK 17 -- overload com lower e upper bound:
random.nextInt(101, 999); // sorteia um numero de quarto de 101 a 998
```

Gerando um Código de Reserva Aleatório

Um padrão comum é restringir a aleatoriedade a um intervalo específico. No Global Stay, o código de confirmação de uma reserva combina duas letras maiúsculas com quatro dígitos. Multiplicando Math.random() (que devolve sempre um double entre 0, inclusive, e 1, exclusivo) por 26 e somando o código ASCII de 'A', chega-se a uma letra maiúscula aleatória:

```
double d = Math.random() * 26;
char letra = (char) ((int) d + 65); // 65 a 90 -> 'A' a 'Z'

// a partir do JDK 17, o mesmo resultado fica mais legível com nextInt:
Random r = new Random();
char letra2 = (char) r.nextInt('A', 'Z' + 1);
```

O que é um Seed?

Na maioria das linguagens, as funções de aleatoriedade não são verdadeiramente aleatórias: existem algoritmos — os geradores pseudoaleatórios de números (PRNGs) — que produzem sequências com aparência de aleatoriedade a partir de um valor inicial, chamado seed.

```
// duas instancias com o MESMO seed produzem a MESMA sequencia:  
Random r1 = new Random(42L);  
Random r2 = new Random(42L);  
System.out.println(r1.nextInt(totalQuartos)); // ex: 57  
System.out.println(r2.nextInt(totalQuartos)); // 57 -- sequencia identica
```

DICA

Um seed fixo é útil para testes determinísticos — por exemplo, simular sempre a mesma sequência de sorteios de upgrade de quarto ao testar o sistema. Sem um seed explícito, o construtor `Random()` usa uma fonte de entropia baseada em tempo, garantindo sequências diferentes a cada execução.

Os Métodos de Stream de Random

Desde o JDK 8, `Random` oferece ints, doubles e longs, cada um com o mesmo conjunto de sobrecargas, prontos para uso com a API de Streams. Chamado sem argumentos, `r.ints()` devolve um `IntStream` infinito — por isso precisa ser combinado com `limit`. A versão de três argumentos já define o próprio tamanho do stream, tornando-o finito automaticamente.

```
Random random = new Random();  
  
// sorteia 5 quartos (entre 100 e 599) para uma promocao de upgrade relampago:  
random.ints(5, 100, 600)  
    .forEach(quarto -> System.out.println("Upgrade liberado: quarto " + quarto));
```

ATENÇÃO

A versão de um único argumento, como `r.ints(10)`, define o tamanho do stream — não um limite superior, como se poderia supor por analogia com `nextInt`. `r.ints(10)` devolve 10 inteiros cobrindo todo o intervalo de int, positivos e negativos, enquanto `r.ints(0, 10)` devolve um stream infinito restrito a esse intervalo. Comportamentos bem diferentes por trás de assinaturas parecidas.

04

BigDecimal — Precisão, Escala e Representação Exata

Precision, scale, unscaled value e por que ponto flutuante falha

Precision define o número total de dígitos de um valor decimal, contando tanto os dígitos à esquerda quanto à direita do ponto. Scale é o número de dígitos à direita do ponto decimal.

Exemplo	Precision	Scale
289.90	5	2
12	2	0
10450.500001	11	6
.075	3	3

Por que Ponto Flutuante Pode Falhar

Muitos números de ponto flutuante conseguem apenas aproximar o decimal que deveriam representar. Pense em $1/3$, uma dízima periódica infinita que precisa ser arredondada em algum ponto. Em cálculos financeiros de larga escala — como dividir a fatura de um pacote corporativo de hospedagem entre vários centros de custo de uma empresa — uma escala insuficiente pode gerar diferenças de centavos difíceis de reconciliar.

ATENÇÃO

Um double pode parecer suficiente para um cálculo isolado, mas ainda assim fica devendo (ou sobrando) centavos ao reconciliar um valor arredondado com um valor calculado internamente com mais casas decimais. Para qualquer valor monetário, BigDecimal é a escolha correta — nunca double ou float.

Como o BigDecimal Armazena um Número

A classe BigDecimal guarda um número em dois campos inteiros: um valor não escalado (unscaled value), do tipo BigInteger, e uma escala (scale), que pode ser positiva, zero ou negativa. Uma escala positiva ou zero define quantos dígitos do valor não escalado ficam após o ponto decimal; uma escala negativa multiplica o valor não escalado por dez elevado à negação da escala.

```

BigDecimal diaria = new BigDecimal("289.90");
diaria.unscaledValue(); // BigInteger 28990
diaria.scale();         // 2
diaria.precision();     // 5

// prefira o construtor de String ao de double, para não herdar
// imprecisões do ponto flutuante binário:
new BigDecimal(0.1);    // 0.1000000000000000055511151231257827021181583404541015625
new BigDecimal("0.1"); // 0.1 -- exato

```

Onde os Centavos se Perdem: um Exemplo do Global Stay

Imagine um pacote corporativo de \$100.000,00 contratado por uma empresa, a ser dividido igualmente entre três filiais que hospedaram funcionários durante um evento. A fração de cada filial é 1 dividido por 3 — uma dízima periódica que qualquer ponto flutuante só consegue aproximar. Calculando essa fração como float e multiplicando pelo valor total, o valor por filial sai defasado em relação ao mesmo cálculo feito com double.

```

double valorPacote = 100_000.00;
int filiais = 3;
float fracaoFloat = 1.0f / filiais;
double fracaoDouble = 1.0 / filiais;

System.out.println(valorPacote * fracaoFloat); // valor com float -- impreciso
System.out.println(valorPacote * fracaoDouble); // valor com double -- melhor, mas ainda
                                                // sobra fracao de centavo na soma

```

Por que o Construtor de String é a Escolha Certa

BigDecimal tem construtores sobrecarregados que aceitam String, double, long, int e BigInteger. Construir a partir de uma String preserva o valor exatamente como escrito; construir a partir de um double herda toda a imprecisão que esse double já carregava antes mesmo de chegar ao BigDecimal. A alternativa recomendada quando se parte de um double é BigDecimal.valueOf, que converte o double para String via Double.toString antes de delegar ao construtor de String.

```

BigDecimal.valueOf(299.0).scale(); // 1 -- Double.toString(299.0) = "299.0"
new BigDecimal("299").scale();     // 0 -- preserva exatamente o que foi escrito

```

Ajustando a Escala com setScale e RoundingMode

Para exibir um valor monetário são normalmente necessárias exatamente 2 casas decimais. O método setScale existe para isso — mas reduzir a escala significa descartar dígitos, e Java não descarta informação silenciosamente: chamar setScale(2) sem especificar como arredondar lança ArithmeticException.

```
BigDecimal taxaServico = new BigDecimal("18.755");
taxaServico.setScale(2); // ArithmeticException: Rounding necessary

taxaServico = taxaServico.setScale(2, RoundingMode.HALF_UP); // 18.76
// BigDecimal é imutável: o resultado precisa ser reatribuído, senão
// o setScale não tem efeito nenhum sobre a variável original.
```

Round ingMo de	Comportamento
HALF_UP	0,5 ou mais no dígito descartado arredonda para cima
HALF_DOWN	0,5 ou mais no dígito descartado arredonda para baixo
HALF_EVEN	Arredonda para o vizinho par (reduz viés em séries longas)
UP	Sempre se afasta de zero
DOWN	Sempre se aproxima de zero (trunca)
CEILING	Sempre arredonda em direção ao infinito positivo
FLOOR	Sempre arredonda em direção ao infinito negativo
UNNECESSARY	Lança exceção se algum arredondamento for de fato necessário

MathContext: Controlando a Precisão do Resultado

Muitos métodos de `BigDecimal` têm uma versão que recebe também um `MathContext`, aplicando regras de precisão e arredondamento ao resultado da operação — e não a um valor já existente. Isso importa em especial na divisão: nem todo quociente tem representação decimal finita.

```
BigDecimal valorPacote = new BigDecimal("100000.00");
BigDecimal fracao = BigDecimal.ONE.divide(BigDecimal.valueOf(3));
// ArithmeticException: Non-terminating decimal expansion

fracao = BigDecimal.ONE.divide(BigDecimal.valueOf(3), MathContext.DECIMAL64); // 16 dígitos
BigDecimal valorPorFilial = valorPacote.multiply(fracao)
    .setScale(2, RoundingMode.HALF_UP);
```

REGRA PRÁTICA

`BigDecimal` é frequentemente descrito como tendo 'precisão arbitrária', porque quem programa decide exatamente qual precisão usar em cada operação — sem ficar limitado aos 6-7 dígitos de um `float`, nem aos 15-16 de um `double`.

Fechando o exemplo do pacote corporativo: multiplicando o valor de cada filial (já normalizado para escala 2) pelo número de filiais, e subtraindo esse total do valor original do pacote, o resto que sobra é exatamente o centavo que não pôde ser dividido igualmente — nem um centavo a mais, nem a menos. É esse resto explícito, e não um erro de arredondamento escondido, que caracteriza um cálculo financeiro correto com `BigDecimal`.

05

Overview: java.time

Temporal, TemporalAccessor, LocalDate/LocalTime/LocalDateTime e pacotes relacionados

Além do pacote java.time propriamente dito, o Java organiza a API de data e hora em pacotes satélites: java.time.temporal e java.time.format, usados com frequência, além de java.time.zone e java.time.chrono, de uso mais especializado.

Pacote	Conteúdo
java.time	LocalDate, LocalTime, LocalDateTime, Instant, Duration, Period, ZonedDateTime
java.time.temporal	Interfaces Temporal/TemporalAccessor, enums ChronoField/ChronoUnit
java.time.format	DateTimeFormatter e estilos de formatação (Full, Long, Medium, Short)
java.time.zone	Regras de fuso horário (uso especializado)
java.time.chrono	Suporte a calendários não-ISO (uso especializado)

LocalDate, LocalTime e LocalDateTime

Essas são as classes mais comuns quando não é necessário incluir informação de fuso horário. No Global Stay, LocalDate representa a data de check-in escolhida pelo hóspede no site, sem se preocupar ainda com o fuso do hotel de destino.

REGRA PRÁTICA

Todos os tipos de java.time seguem o mesmo vocabulário de prefixos de método: at (combina temporais, ex: atStartOfDay), get (extrai um campo, ex: getYear), is (predicado de comparação, ex: isAfter) e with (retorna uma cópia com um campo alterado). Reconhecer esse padrão evita ter que memorizar a API de cada classe individualmente.

```
LocalDate checkIn = LocalDate.of(2026, 8, 14);
LocalDate checkOut = checkIn.plus(3, ChronoUnit.DAYS);
checkIn.isAfter(checkOut);           // false
checkIn.getDayOfWeek();              // sexta-feira
checkIn.format(DateTimeFormatter.ISO_DATE);
```

Criando Instâncias

now() devolve o momento atual; várias sobrecargas de of() criam instâncias explícitas; parse() interpreta uma String formatada, normalmente no padrão ISO.

```

    LocalDate.now();
    LocalDate.of(2026, Month.AUGUST, 14);
    LocalDate.parse("2026-08-14");

    LocalTime horarioPadraoCheckIn = LocalTime.of(14, 0);
    LocalDateTime.of(checkIn, horarioPadraoCheckIn);

```

Armazenamento Interno

Classe	Campos Internos
LocalDate	int para o ano; short para mês e dia
LocalTime	byte para hora, minuto e segundo; int para nanossegundos
LocalDateTime	um LocalDate + um LocalTime

Campos Armazenados vs. Campos Calculados

Nem todo getter devolve um valor de fato guardado internamente. `getDayOfMonth` devolve o dia armazenado diretamente; já `getDayOfWeek` e `getDayOfYear` são calculados a partir do ano, mês e dia, usando um algoritmo interno de calendário — uma distinção transparente para quem usa a API, mas que explica por que `LocalDate` responde a perguntas que vão muito além dos três campos que efetivamente guarda.

```

    LocalDate checkIn = LocalDate.of(2026, 8, 14);
    checkIn.getDayOfMonth(); // 14 -- valor armazenado diretamente
    checkIn.getDayOfWeek(); // FRIDAY -- calculado a partir de ano+mes+dia
    checkIn.getDayOfYear(); // 226 -- calculado contando desde 1 de janeiro

```

with, plus e minus: Ajustando uma Data sem Alterar a Original

Como toda classe de `java.time` é imutável, não existem setters. Os métodos `with` têm o comportamento que um setter teria, mas sempre devolvem uma nova instância. Já `plus` e `minus` somam ou subtraem uma quantidade a um campo, em vez de defini-lo diretamente.

```

    LocalDate checkIn = LocalDate.of(2026, 8, 14);
    checkIn.withYear(2027); // 14 de agosto de 2027 -- nova instancia
    checkIn.plusDays(3); // 17 de agosto de 2026 -- nova estadia de 3 noites
    // a variavel original "checkIn" permanece intacta em todos os casos

```

Comparando Datas e o Método range

`compareTo`, herdado de `Comparable`, permite ordenar reservas por data. `isAfter` e `isBefore` expressam a mesma comparação de forma mais legível. O método `range`, herdado de `TemporalAccessor`, devolve os limites válidos de um campo para aquela instância específica — útil para validar um valor antes de usá-lo em um `with`.

Nem Todo Campo Serve para Toda Classe

`LocalTime` não tem noção de ano, mês ou dia. Chamar `get(ChronoField.YEAR)` em um `LocalTime` compila normalmente, mas lança `UnsupportedTemporalTypeException` em tempo de execução, porque esse campo simplesmente não existe naquele tipo de objeto.

```
LocalTime horarioCheckIn = LocalTime.of(14, 0);
horarioCheckIn.get(ChronoField.YEAR);           // UnsupportedTemporalTypeException
horarioCheckIn.plus(1, ChronoUnit.DAYS);         // UnsupportedTemporalTypeException
horarioCheckIn.plus(24, ChronoUnit.HOURS);       // OK -- resultado: mesmo horario (14:00)
```

datesUntil: um Stream de Datas

A partir do JDK 9, `LocalDate` ganhou `datesUntil`, que devolve um `Stream` entre a instância que chama o método (inclusive) e um `LocalDate` limite (exclusivo) — útil, por exemplo, para listar todas as noites de uma estadia, uma a uma.

```
LocalDate checkIn = LocalDate.of(2026, 8, 14);
LocalDate checkOut = checkIn.plusDays(5);

checkIn.datesUntil(checkOut)
    .forEach(noite -> System.out.println("Diária referente a: " + noite));
```

06

TemporalField vs. TemporalUnit

ChronoField, ChronoUnit e a diferença entre campo e unidade de tempo

Um TemporalField representa um campo específico dentro de uma data-hora, como MONTH_OF_YEAR ou DAY_OF_WEEK — a parte do mês em uma data, por exemplo. Já um TemporalUnit representa uma unidade ou duração de tempo, como DAYS ou MONTHS — não uma parte da data, mas a quantidade de tempo usada para medir o intervalo entre duas datas-horas.

Interface	Representa	Exemplos de Constantes
TemporalField	Um campo específico dentro de uma data-hora	ChronoField.MONTH_OF_YEAR, DAY_OF_WEEK
TemporalUnit	Uma unidade/duração usada para medir intervalos	ChronoUnit.YEARS, MONTHS, DAYS

Qual Método Usa Qual Interface

Os métodos get e with recebem um TemporalField, porque operam sobre um campo específico. Já plus, minus e until recebem um TemporalUnit, porque operam sobre uma quantidade de tempo. No contexto do Global Stay, isso aparece ao consultar em qual mês cai um check-in (get) e ao calcular quantas noites faltam para uma reserva começar (until).

```
LocalDate checkIn = LocalDate.of(2026, 9, 1);

// get / with usam TemporalField (ChronoField):
int mesDoCheckIn = checkIn.get(ChronoField.MONTH_OF_YEAR); // 9
LocalDate checkInAjustado = checkIn.with(ChronoField.DAY_OF_MONTH, 15);

// plus / minus / until usam TemporalUnit (ChronoUnit):
LocalDate proximaJanela = checkIn.plus(2, ChronoUnit.MONTHS);
long noitesAteACheckIn = LocalDate.now().until(checkIn, ChronoUnit.DAYS);
```

DICA

A assinatura do método é a pista mais confiável: se ele pergunta 'qual valor tem este campo' (get/with), use ChronoField. Se pergunta 'quanto tempo passou ou vai passar' (plus/minus/until), use ChronoUnit. Confundir os dois gera erro de compilação, já que TemporalField e TemporalUnit não são intercambiáveis.

07

Instant, Epoch Time e Fusos Horários

Timeline, Instant, Epoch, GMT/UTC e a TZDB da IANA

Um ponto em uma linha do tempo — o instante exato em que um hóspede confirma o pagamento de uma reserva — é chamado de Instant. Um intervalo nessa mesma linha, medido em unidades de calendário, é um Period; medido em unidades de relógio, é uma Duration.

REGRA PRÁTICA

Um Instant marca um ponto único na linha do tempo; um Period mede um intervalo em unidades de calendário (anos, meses, dias); e uma Duration mede um intervalo em unidades de relógio (horas, minutos, segundos). As três ideias descrevem a mesma linha do tempo, mas em granularidades diferentes.

A Classe Instant

Instant representa apenas um ponto no tempo, um timestamp. Internamente armazena segundos e nanossegundos a partir do epoch fixo de 1970-01-01Z. Por isso não pode ser formatado como data ou hora sem informação de fuso — o sufixo Z indica referência a UTC.

```
// registrando o instante exato em que um pagamento foi confirmado no Global Stay:
Instant pagamentoConfirmado = Instant.now();
System.out.println(pagamentoConfirmado); // ex: 2026-08-01T18:42:07.123456Z

long segundos = pagamentoConfirmado.getEpochSecond();
int nanos = pagamentoConfirmado.getNano();
```

Essa uniformidade — sempre referenciado a UTC, sem depender do fuso do sistema — permite comparar dois Instants de forma confiável, mesmo que um pagamento tenha sido processado no servidor de Lisboa e outro no servidor de Tóquio.

GMT, UTC e a Origem dos Fusos Horários

O Meridiano de Greenwich foi historicamente acordado como referência zero para medições de longitude. O horário em Greenwich era originalmente tempo solar; com relógios atômicos, tornou-se possível um horário mais preciso — o UTC, que substituiu o GMT em 1972. As diferenças entre os dois podem chegar a 0,9 segundos por dia, por isso costumam ser usados como sinônimos quando grande precisão não é exigida.

Um fuso horário é composto por um offset em relação a UTC e, opcionalmente, informação de horário de verão. Java obtém seus dados de fuso da TZDB da IANA, sua fonte padrão.

O Formato dos Identificadores de Fuso

A TZDB identifica cada fuso por uma String Continente/Cidade — por exemplo, "Asia/Tokyo" ou "America/New_York". Usar uma cidade específica, em vez de um offset fixo, é o que permite ao Java resolver automaticamente as regras de horário de verão daquela região.

```
ZoneId hotelLisboa = ZoneId.of("Europe/Lisbon");
ZoneId hotelToquio = ZoneId.of("Asia/Tokyo");
ZoneId hotelNovaYork = ZoneId.of("America/New_York");
ZoneId hotelSydney = ZoneId.of("Australia/Sydney");
```

A Classe ZoneId na Prática

ZoneId.systemDefault() devolve o fuso que a JVM está usando no momento.

ZoneId.getAvailableZoneIds() devolve um Set com mais de 600 identificadores suportados — um número maior do que os 24 fusos de uma hora inteira sugeririam, já que existem deslocamentos de meia hora e até de 45 minutos em algumas regiões.

```
ZoneId aqui = ZoneId.systemDefault();
Set<String> todasAsZonas = ZoneId.getAvailableZoneIds();
todasAsZonas.stream()
    .filter(id -> id.startsWith("Asia"))
    .sorted()
    .forEach(System.out::println);
```

ATENÇÃO

Identificadores de fuso nunca contêm espaço — onde o nome de uma cidade teria um espaço, o identificador usa underscore, como em "America/New_York". Códigos curtos legados, como "EST", ainda funcionam em algumas APIs antigas, mas seu uso é desencorajado por serem ambíguos entre países.

Alterando o Fuso Horário Padrão da JVM

É possível alterar o fuso padrão de toda a aplicação com System.setProperty, passando "user.timezone". Essa chamada precisa acontecer antes de qualquer código que manipule datas, porque o fuso padrão é armazenado em cache assim que é consultado pela primeira vez.

```
public static void main(String[] args) {
    System.setProperty("user.timezone", "Europe/Lisbon");
    // qualquer código de data-hora a partir daqui já considera esse fuso
    LocalDateTime agora = LocalDateTime.now();
}
```

System.currentTimeMillis e System.nanoTime

Método	Retorna	Uso Recomendado
currentTimeMillis()	Milissegundos desde a epoch, baseado no SO	Medir tempo decorrido ou fornecer timestamps
nanoTime()	Nanossegundos a partir de uma origem arbitrária	Medir tempo decorrido dentro de uma JVM — nunca como timestamp

08

ZonedDateTime, Duration, Period e ChronoUnit.between

Data-hora com fuso horário, TemporalAmount e cálculo de intervalos

Enquanto LocalDateTime representa uma data-hora sem contexto de fuso, ZonedDateTime combina um LocalDateTime com um ZoneId, representando um instante específico e não ambíguo. No Global Stay, isso é essencial para converter o horário de chegada informado por um hóspede de Nova York para o horário local do hotel em Sydney.

```
ZonedDateTime chegadaNY = ZonedDateTime.of(
    LocalDateTime.of(2026, 8, 14, 22, 30), ZoneId.of("America/New_York"));
ZonedDateTime chegadaSydney = chegadaNY.withZoneSameInstant(ZoneId.of("Australia/Sydney"));

System.out.println(chegadaNY);
// 2026-08-14T22:30-04:00[America/New_York]
System.out.println(chegadaSydney);
// 2026-08-15T12:30+10:00[Australia/Sydney] -- mesmo instante, fuso diferente
```

ATENÇÃO

withZoneSameInstant recalcula a hora local para o novo fuso, mantendo o mesmo ponto exato na linha do tempo — o método certo para converter um horário de chegada entre fusos. Já withZoneSameLocal mantém os mesmos números de data-hora, mas os reinterpreta em outro fuso, resultando em um instante diferente. Escolher o método errado é a causa mais comum de bugs de agendamento entre fusos.

Convertendo um Instant em Data-Hora Local

Um Instant sozinho não carrega fuso horário. Para exibir esse instante como uma data-hora legível em um fuso específico, é preciso combiná-lo com um ZoneId. ZonedDateTime.ofInstant faz essa conversão mantendo a informação de qual fuso foi usado.

```
Instant pagamentoConfirmado = Instant.parse("2026-08-01T23:50:00Z");
ZonedDateTime localLisboa = ZonedDateTime.ofInstant(
    pagamentoConfirmado, ZoneId.of("Europe/Lisbon"));
ZonedDateTime localToquio = ZonedDateTime.ofInstant(
    pagamentoConfirmado, ZoneId.of("Asia/Tokyo"));
// o mesmo Instant produz datas de calendario diferentes conforme o fuso escolhido
```

Consultando Regras de Horário de Verão

Cada ZoneId carrega um conjunto de regras, acessível via getRules(), que descreve o offset UTC e o comportamento de horário de verão ao longo do tempo.

```
ZoneRules regrasSydney = ZoneId.of("Australia/Sydney").getRules();
Instant agora = Instant.now();
regrasSydney.isDaylightSavings(agora); // true ou false, conforme a época do ano
regrasSydney.getOffset(agora); // +10:00 ou +11:00
```

TemporalAdjuster: Ajustes Predefinidos

Além da versão de `with` que recebe um campo e um valor, existe uma sobrecarga que recebe um `TemporalAdjuster`. A classe utilitária `TemporalAdjusters` reúne métodos estáticos prontos para ajustes comuns, como o primeiro dia do próximo mês ou a próxima ocorrência de um dia da semana — útil para calcular automaticamente a data de fechamento mensal de faturas do Global Stay.

```
LocalDate hoje = LocalDate.now();
LocalDate fechamentoFatura = hoje.with(TemporalAdjusters.lastDayOfMonth());
LocalDate proximaSegunda = hoje.with(TemporalAdjusters.next(DayOfWeek.MONDAY));
```

TemporalAmount: Duration e Period

`Duration` e `Period` não implementam `Temporal` nem `TemporalAccessor` — implementam `TemporalAmount`, representando quantidades de tempo, não pontos no tempo.

Classe	Interface	Unidades Típicas
Instant	<code>Temporal</code> / <code>TemporalAccessor</code>	segundos + nanossegundos desde a epoch
Duration	<code>TemporalAmount</code>	horas, minutos, segundos
Period	<code>TemporalAmount</code>	anos, meses, dias

```
// Duration -- baseada em unidades de tempo:
LocalTime checkIn = LocalTime.of(14, 0);
LocalTime chegadaReal = LocalTime.of(19, 45);
Duration atrasoCheckIn = Duration.between(checkIn, chegadaReal); // PT5H45M

// Period -- baseado em unidades de data:
LocalDate inicioEstadia = LocalDate.of(2026, 8, 14);
LocalDate fimEstadia = LocalDate.of(2026, 8, 20);
Period duracaoEstadia = Period.between(inicioEstadia, fimEstadia); // P6D
```

ChronoUnit.between — Alternativa Genérica

Além de `Duration.between` e `Period.between`, `ChronoUnit` oferece o método estático `between`, útil quando se precisa de um único número inteiro em vez de um objeto `Duration` ou `Period` completo — por exemplo, o total de noites de uma reserva.

```
long noites = ChronoUnit.DAYS.between(inicioEstadia, fimEstadia); // 6
```

REGRA PRÁTICA

Use `Period` ou `Duration` quando precisar de um objeto rico que descreva o intervalo em múltiplas unidades ao mesmo tempo. Use `ChronoUnit.between` quando precisar de um único valor numérico em uma unidade específica. As três abordagens calculam a mesma coisa de formas diferentes.

09

Localization — Introdução ao Locale

A classe `Locale`, seus campos e o suporte nativo do Java

Onde uma pessoa está — ou de onde ela vem — determina como espera ver datas, moedas e números formatados. No `Global Stay`, um mesmo valor de diária precisa aparecer como "R\$ 950,00" para um hóspede brasileiro e como "¥ 950" arredondado, sem casas decimais, para um hóspede japonês.

A Classe `Locale`

`Locale` é o nome de uma classe do pacote `java.util` que dá suporte tanto à localização quanto à internacionalização. Um `Locale` tem cinco campos: idioma, país (ou região), variante, script e extensões.

Campo	Descrição
<code>language</code>	Código ISO 639 do idioma (ex: "pt", "ja")
<code>country / region</code>	Código ISO 3166 do país (ex: "BR", "JP")
<code>variant</code>	Variação específica de um <code>Locale</code> (uso menos comum)
<code>script</code>	Sistema de escrita, quando relevante (ex: "Latn")
<code>extensions</code>	Extensões BCP 47 adicionais (ex: calendário, ordenação)

```
Locale hospedeBR = new Locale("pt", "BR");
Locale hospedeJP = Locale.JAPAN; // constante pre-definida
Locale hospedeAU = Locale.forLanguageTag("en-AU");

hospedeBR.getDisplayName(); // "português (Brasil)"
hospedeBR.getCountry();    // "BR"
```

O `Locale` Padrão: `getDefault` e `setDefault`

`Locale.getDefault()` devolve o `Locale` que a JVM está usando, normalmente derivado do sistema operacional. `Locale.setDefault` sobrescreve esse valor a partir desse ponto.

Constantes Prontas e `Locale.Builder`

A classe `Locale` não tem construtor sem argumentos — no mínimo é preciso um idioma. Para os casos mais comuns já existem campos estáticos prontos (`Locale.US`, `Locale.JAPAN`). Quando nenhuma constante atende, `Locale.Builder` permite montar a instância de forma fluente.

```
Locale hospedeIndia = new Locale.Builder()
    .setLanguage("en")
    .setRegion("IN")
    .build();
```

Suporte Nativo de Java para Localização

O Java formata datas, números e moedas sem esforço adicional, além de definir um Locale e passá-lo a determinados métodos.

```
LocalDate checkIn = LocalDate.of(2026, 8, 14);

DateTimeFormatter fmtBR = DateTimeFormatter
    .ofLocalizedDate(FormatStyle.FULL)
    .withLocale(new Locale("pt", "BR"));
DateTimeFormatter fmtJP = DateTimeFormatter
    .ofLocalizedDate(FormatStyle.FULL)
    .withLocale(Locale.JAPAN);

System.out.println(checkIn.format(fmtBR)); // sexta-feira, 14 de agosto de 2026
System.out.println(checkIn.format(fmtJP)); // 2026年8月14日

NumberFormat moedaBR = NumberFormat.getCurrencyInstance(new Locale("pt", "BR"));
System.out.println(moedaBR.format(950.0)); // R$ 950,00
```

Padrões Customizados com DateTimeFormatter.ofPattern

Quando nenhum FormatStyle predefinido atende, ofPattern aceita uma String de padrão montada a partir de letras específicas: E para dia da semana, M para mês, d para dia, y para ano. A quantidade de letras repetidas controla o nível de detalhe.

```
DateTimeFormatter confirmacaoReserva = DateTimeFormatter.ofPattern("EEEE, MMMM d, yyyy");
System.out.println(checkIn.format(confirmacaoReserva.withLocale(Locale.FRANCE)));
// vendredi, août 14, 2026 -- dia da semana e mes por extenso, em frances
```

NumberFormat e Currency

NumberFormat.getCurrencyInstance(locale) formata um valor como moeda, arredondando automaticamente para o número de casas decimais que faz sentido para aquela moeda — o iene japonês, por exemplo, não usa casas fracionárias.

```
NumberFormat moedaJP = NumberFormat.getCurrencyInstance(Locale.JAPAN);
System.out.println(moedaJP.format(950.50)); // ¥951 -- arredondado, sem casas decimais

Currency real = Currency.getInstance(new Locale("pt", "BR"));
real.getCurrencyCode(); // BRL
real.getDisplayName(Locale.US); // "Brazilian Real"
```

Scanner e Locale: Lendo Entrada Localizada

Um `BigDecimal` construído a partir de uma `String` só aceita um único ponto decimal; um separador de milhar no meio do texto quebra o construtor. A classe `Scanner` sabe interpretar entrada já formatada de acordo com um `Locale` através de `useLocale`.

```
Scanner scanner = new Scanner(System.in);
scanner.useLocale(new Locale("pt", "BR")); // Brasil: ponto agrupa, virgula e decimal
// entrada digitada pelo atendente: 1.250,50
BigDecimal valorReserva = scanner.nextBigDecimal(); // interpretado como 1250.50
```

ATENÇÃO

Sem chamar `useLocale`, `Scanner` usa o `Locale` padrão da JVM para interpretar números. Se o `Locale` configurado não corresponder ao formato realmente digitado, `nextBigDecimal` lança `InputMismatchException`, mesmo que a entrada pareça um número perfeitamente válido em outro `Locale`.

Internacionalização, ou `I18n`, é o método de projetar uma aplicação para que elementos de idioma e regionais sejam plug-and-play. Strings de mensagens e de interface ficam armazenadas fora do código, recuperáveis a partir de um `Locale` — tema do próximo capítulo.

10

Internationalization — ResourceBundle

Arquivos .properties, bundles, matching de Locale e alternativas

ResourceBundle é uma classe abstrata do pacote java.util. Uma instância é obtida chamando um dos métodos estáticos getBundle. A abordagem mais comum depende de arquivos .properties.

O Arquivo .properties

Um arquivo .properties é um arquivo de texto simples com pares chave-valor. O nome do arquivo combina um nome base com alguma parte do identificador de Locale, terminando com a extensão .properties. No Global Stay, as mensagens de confirmação de reserva ficam em um bundle chamado ReservaMensagens:

```
# ReservaMensagens.properties (arquivo base, sem sufixo de locale)
confirmacao=Reservation confirmed
checkin=Check-in

# ReservaMensagens_pt_BR.properties (variação para português do Brasil)
confirmacao=Reserva confirmada
checkin=Check-in

# ReservaMensagens_ja.properties (variação para japones)
confirmacao=■■■■■
checkin=■■■■■■■
```

```
ResourceBundle bundle = ResourceBundle.getBundle(
    "ReservaMensagens", new Locale("pt", "BR"));
System.out.println(bundle.getString("confirmacao")); // "Reserva confirmada"

ResourceBundle bundleJP = ResourceBundle.getBundle("ReservaMensagens", Locale.JAPAN);
System.out.println(bundleJP.getString("confirmacao")); // "■■■■■"
```

O Processo de Matching do Java

Java monta uma sequência de nomes de arquivo candidatos, do mais específico ao menos específico, até sobrar apenas o nome base do bundle. O arquivo base funciona como última rede de segurança.

REGRA PRÁTICA

Essa cadeia de fallback também se aplica por chave individual, não apenas por arquivo inteiro: se uma chave existir no arquivo base mas não em uma variação mais específica de Locale, getString ainda encontra o valor no arquivo base, sem lançar exceção. Isso poupa bastante trabalho de duplicação quando uma variação regional difere do idioma principal em apenas algumas poucas palavras.

ATENÇÃO

As chaves de um arquivo `.properties` são sensíveis a maiúsculas e minúsculas — `getString` não ignora essa diferença. Uma chave ausente, por erro de digitação ou por realmente não existir em nenhum arquivo da cadeia de fallback, faz `getString` lançar `MissingResourceException`.

Múltiplos Bundles em uma Mesma Aplicação

Nada impede usar mais de um `ResourceBundle` simultaneamente — é comum separar bundles por área de responsabilidade, como um bundle para mensagens de reserva e outro para rótulos de botões da interface.

```
ResourceBundle mensagens = ResourceBundle.getBundle("ReservaMensagens", locale);
ResourceBundle botoes = ResourceBundle.getBundle("InterfaceBotoes", locale);

String textoConfirmacao = mensagens.getString("confirmacao");
String rotuloConfirmar = botoes.getString("btn.confirmar");
```

Parametrizando Mensagens com formatted

Uma mensagem obtida de um `ResourceBundle` costuma precisar de dados dinâmicos inseridos nela — o nome do hóspede, o número de noites. O método `formatted`, da própria classe `String`, permite tratar o texto como um molde e substituir marcadores de posição.

```
# no arquivo .properties:
# boasVindas=Ola, %s! Sua reserva tem %d noite(s) confirmada(s).

String mensagem = mensagens.getString("boasVindas")
                    .formatted(nomeHospede, totalNoites);
```

Alternativas ao Arquivo .properties

É possível estender `ResourceBundle` diretamente, ou a classe abstrata `ListResourceBundle`, quando os dados não são simples `Strings` — por exemplo, quando o recurso precisa ser um objeto, array ou outra estrutura complexa. Outros formatos, como XML, também podem ser usados com configuração extra, estendendo `ResourceBundle.Control`.

DICA

As chaves de ambos os arquivos de propriedades ficam, por convenção, em inglês, e são idênticas entre os arquivos. O idioma escolhido para as chaves é uma decisão livre — mas elas precisam ser consistentes entre arquivos, para que a mesma chave possa ser usada para consultar o valor em qualquer variação de idioma.



Fim de Apostila

Com Math, Random, BigDecimal, o pacote java.time e o suporte de Java para localização e internacionalização dominados, você fecha um conjunto de utilitários de uso diário que aparecem em praticamente todo sistema real — de cálculos financeiros a agendamento entre fusos horários e interfaces multi-idioma.

Continue praticando com cenários próprios: são exatamente esses detalhes de precisão numérica, fusos horários e localização que separam um protótipo de um sistema pronto para o mundo real.